

UNIVERSITY OF CLAUDE · COURSE SYLLABUS

CSCE 717 — Algorithmic Game Theory

The input has interests, so correctness must include the incentive to report honestly

AT A GLANCE

Credits: 3 credit-equivalents · **Modules:** 13 · **Term:** Semester 12 (with CSCE 640 and CSCE 628)

Prerequisites: CSCE 629 Analysis of Algorithms; CSCE 669 Computational Optimization for linear programming and duality; CSCE 637 Complexity Theory for Module 03; CSCE 642 Deep Reinforcement Learning is useful for Module 12 and is not required

Effort: 12–14 hours per week · 13 modules + 2 project weeks

Deliverable: A mechanism for a resource allocation problem you choose — with its incentive properties proved or its failure of them demonstrated by a concrete manipulation, its approximation ratio stated, and its behaviour simulated against strategic rather than honest agents

Course description

Every algorithms course before this one assumed the input was given. Module 01 removes that assumption — the input is now reported by people who want particular outputs — and everything that follows is about what that changes, which turns out to be the definition of correctness itself.

The first third is the analysis side. Games, equilibria, and what Nash's theorem does and does not promise (Module 02); the complexity of actually finding an equilibrium (Module 03), which is hard in a precise and interesting sense; and the price of anarchy (Modules 04 and 05) — the quantitative cost of letting agents choose for themselves.

The second third is the design side, and it is the useful one. Mechanism design asks what outcomes can be implemented when truthful reporting has to be in each agent's interest (Module 06) — and auctions are where that theory is sharpest and most deployed (Modules 07 and 08), with matching and fair division covering the cases where money is unavailable (Modules 09 and 11).

The third theme is the impossibility results, which are load-bearing rather than decorative. Arrow and Gibbard-Satterthwaite say that no voting rule has all the properties you want (Module 10) — so every real system is a choice about which property to give up, and knowing which is the whole of being able to evaluate one.

And the closing position is about claims. Module 13 asks what it means to say a mechanism works — because 'incentive compatible' is a statement about a model of the agents, and the agents are real people who may collude, misunderstand the rules, or care about things the model does not include.

Learning outcomes

On finishing this course you should be able to do the following without reference material.

- 1 Explain why strategic inputs change what correctness means.
- 2 Define games and equilibria, and state Nash's theorem precisely.
- 3 Explain the complexity of computing equilibria.
- 4 Define and compute the price of anarchy.
- 5 Analyse routing and congestion games.
- 6 State the mechanism design problem and incentive compatibility.
- 7 Derive the second-price auction and revenue equivalence.
- 8 Explain combinatorial auctions and their approximation.
- 9 Design and analyse matching mechanisms.
- 10 State the social choice impossibility results and what follows.
- 11 Explain fair division and its fairness notions.
- 12 Explain no-regret dynamics and what they converge to.
- 13 Claim that a mechanism works, honestly.

Module schedule

One module per week for thirteen weeks, then two weeks for the final project. Each module is a slide deck, a set of lecture notes, mapped external lectures, and exercises that must run.

Wk	Module	Core question it answers
1	When the Input Has Interests	Who gave you this input, and what do they want?
2	Games and Equilibria	What does it mean to predict what agents will do?
3	Computing Equilibria	If it always exists, why can't you find it?
4	The Price of Anarchy	How much worse is the equilibrium than the optimum?
5	Routing and Congestion	Can adding a road make everyone slower?
6	Mechanism Design	Can you always make truth-telling the best strategy?
7	Single-Item Auctions	Why does the second-highest bid set the price?
8	Combinatorial Auctions	What happens when approximating the allocation destroys truthfulness?
9	Matching	How do you allocate when you cannot charge?
10	Voting and Social Choice	Is there a fair voting rule?

Wk	Module	Core question it answers
11	Fair Division	What exactly do you mean by fair?
12	Learning in Games	If nobody can compute the equilibrium, what happens instead?
13	Claiming a Mechanism Works	'It's incentive compatible.' Under what model of the agents?

Projects

Project 1 — Measure the price of anarchy (after Module 6)

Build a congestion game, find its equilibria, and measure what selfish routing costs.

Requirements

- **A routing or congestion game implemented**, with latency functions you choose and state.
- **The social optimum computed** — by optimisation, exactly — and **the equilibrium found by best-response dynamics**.
- **The ratio measured across many random instances**, with the distribution reported rather than the worst case alone.
- **Braess's paradox reproduced: an added edge that makes everybody worse off**, constructed and verified.
- **And the theoretical bound compared** to what you measured — with the gap explained.
- **Plus: does best-response dynamics always converge** in your game, and why?

Done means

- **Braess's paradox constructed and verified** — **which is the deliverable that shows you understand equilibrium rather than optimisation**.
- **The distribution of ratios reported**, not only the maximum — because **the worst case is a bound and the typical case is the engineering question**.
- **The convergence question answered with a reason** (a potential function, or a counterexample) rather than by observation.
- **And the latency functions stated**, since **the price of anarchy bound depends on the class of functions allowed** (Module 04 §3), which is the result's most misquoted condition.

Project 2 — Design a mechanism, then attack it (after Module 12)

Build an allocation mechanism and find out what strategic agents do to it.

Requirements

- **An allocation problem you choose** — scheduling, bandwidth, course seats, shared compute — **with the agents, their preferences, and the designer's objective stated precisely**.
- **A mechanism specified**: allocation rule and payment rule, or an ordinal rule if money is unavailable.
- **Its incentive properties proved, or its failure demonstrated by a concrete manipulation** — **a specific agent, specific true preferences, and a specific profitable misreport**.
- **The efficiency or approximation ratio stated**, with the benchmark named.
- **Simulated against strategic agents** (Module 12's learners), not honest ones — **and the outcome compared to the honest-agent case**.

- **And the honest claim** in Module 13 §3's form, including which impossibility result your design is choosing to live with.

Done means

- **The manipulation concrete — named agent, named misreport, computed gain** — because **'it is manipulable' without an exhibit is not a finding** (Module 13 §2).
- **The strategic simulation run** and compared to the honest baseline, which is the point of the project.
- **The impossibility named: every mechanism gives something up** (Modules 06 §4, 10 §3), and saying which is what makes a design defensible.
- **And the benchmark stated**, because **an approximation ratio against an unstated optimum is not a number** — which is CSCE 629's discipline in a new setting.

Assessment

Nobody grades this. The standard is therefore mechanical, and you apply it to yourself honestly or the credential is worth nothing.

Component	Weight	Standard
Module exercises	40%	Code runs and produces the specified output. Not 'I understand how I would do it.'
Projects	45%	In the portfolio repository, with a README showing results and a written note on what was hard.
Self-checks	15%	Answered in writing from memory. Recognition is not knowledge; writing exposes the difference.

Required materials

All free. Nothing in this course requires a purchase.

Resource	Role
Nisan, Roughgarden, Tardos & Vazirani — Algorithmic Game Theory (free PDF from the authors)	The primary text, and it is free. Modules 02 through 11 follow its organisation, and the chapters are by the people who proved the results.
Roughgarden — Twenty Lectures on Algorithmic Game Theory, with the free video lectures and notes	The best entry point, free in full. Modules 01, 04, 06, and 07 especially — and the lectures are worth watching rather than only reading.
Shoham & Leyton-Brown — Multiagent Systems (free PDF)	Modules 02, 10, and 12, free. Broader than the algorithmic texts, and the social choice chapters are excellent.

Resource	Role
Krishna — Auction Theory	Module 07. The economics treatment, where revenue equivalence is derived properly. Library copy.
Roth — Who Gets What and Why	Module 09. Matching markets as actually deployed — kidney exchange, school choice, medical residencies — by the person who built several of them.
Brandt, Conitzer, Endriss, Lang & Procaccia — Handbook of Computational Social Choice (free PDF)	Modules 10 and 11, free. The impossibility results and the fair division literature, rigorously.

Tooling

- **Pen and paper for the proofs** — **most of this course is theorems**, and **Modules 06 and 07's arguments have to be worked by hand** before any implementation is meaningful.
- **A linear programming solver** — **any of them, and the duality from CSCE 669 §05 is used directly** in Modules 03 and 08.
- **Python for simulation**, because **the interesting question in Project 2 is what strategic agents do to your mechanism**, and that is answered by simulating them rather than by assuming they are honest.
- **Gambit or a similar game solver** for small normal-form games — useful in Module 02 and quickly outgrown, which is itself Module 03's point.
- **And a best-response or no-regret learner** (Module 12) — **the cheapest way to find out whether your mechanism survives agents who adapt**, which is Project 2's requirement.
- **Plus, for Project 2, a concrete manipulation** — **a specific misreport that makes a specific agent better off** — because **'it is manipulable' is a claim that needs an exhibit** (Module 13 §2).

Academic honesty, restated for a solo student

There is no one to cheat but yourself, which makes it easier, not harder. Two rules: copy code only after you have written a worse version yourself, and never let a working result stand in for an understood one. When you paste something you do not understand, write a TODO naming what you do not understand, and go back to it.